

Lab 04

Process Management

Course: Operating System

Class: Computer Science IV

Instructor: Bilal Ahmed

Learning Objectives:

- Understand processes in practice.
- Learn about process types.
- Tools to monitor processes.
- Killing a process.
- Priority in Process

Process Management: Refers to process monitoring, management, creation, termination and scheduling. It has great importance in operating systems as it allows you to have parallel processing and efficient use of shared resources. It is responsible for sharing resources such as CPU time, memory and peripheral devices efficiently between the multiple processes. Its basic aim is to maximize the CPU throughput and minimize the response time.

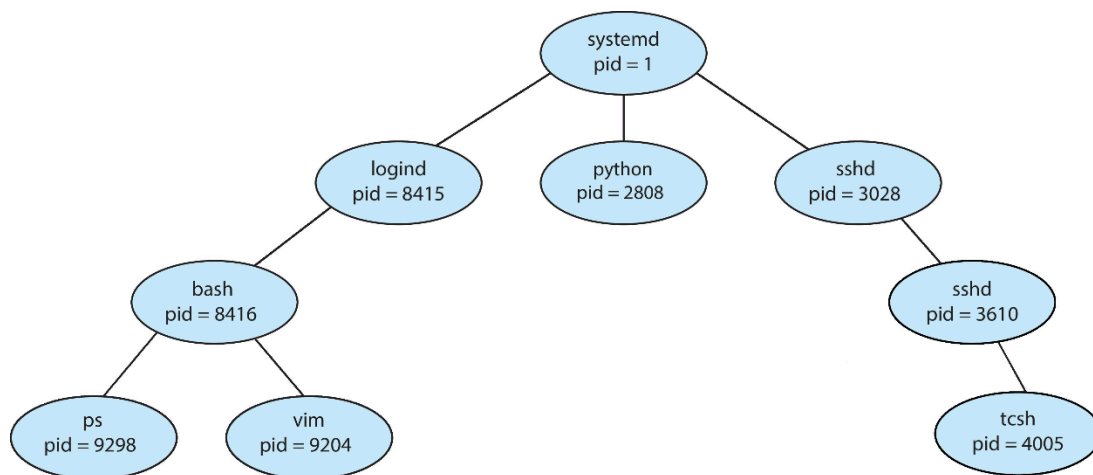
Process Management in Linux: Linux provides various robust command line tools such as ps, top, htop, pstree, etc to monitor and manage the process running in operating system, whereas Windows and MacOS users often use GUI built-in tools like task manager and activity manager respectively.



PS – Process Status: It is one of the most basic tools of Linux to monitor the basic details of the process including its, process id, session id, terminal, command, memory and CPU usage and much more. Execute the ps to see processes running in current terminal.

```
bilal@bilal: ~  
bilal@bilal:~$ ps  
  PID TTY          TIME CMD  
 75405 pts/0    00:00:00 bash  
 75999 pts/0    00:00:00 ps  
bilal@bilal:~$
```

Process ID (PID): A distinct identifier is assigned to each process, enabling its unique identification within the system. General tree of process of pids is given in following picture (taken from book).



Terminal Type (TTY): The type of terminal to which a user is currently logged in, specifying the communication interface for interaction.

CPU Time (TIME): The duration, measured in minutes and seconds, that a process has utilized the central processing unit (CPU) since its initiation.

Command Name (CMD): The label representing the name of the command that initiated the process, providing insight into the executed action.

Process States: In operating systems, process usually experiences some states from initiation to termination as illustrated below from the following figure from the book.

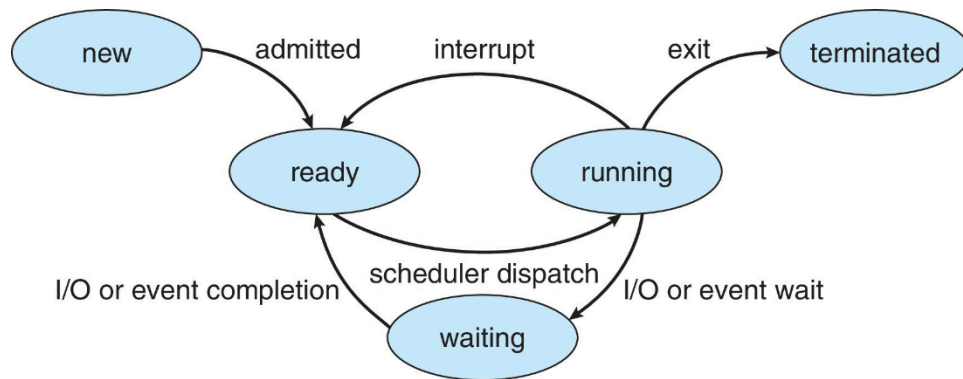
New: The process is being created

Running: Instructions are being executed

Waiting: The process is waiting for some event to occur

Ready: The process is waiting to be assigned to a processor

Terminated: The process has finished execution



Process States in Linux: Linux follows the following symbolic representation for process states.

- **S:** Sleeping - Process is waiting for an event to complete.
- **R:** Running - Process is currently executing or ready to execute.
- **D:** Uninterruptible sleep - Process is waiting for a resource that cannot be interrupted, like a disk I/O operation.
- **T:** Stopped - Process has been halted, usually by receiving a signal, and can be restarted later.
- **Z:** Zombie - Process has completed execution, but its entry remains in the process table.
- **I:** Process is multithreaded.
- **s:** Process is a session leader.
- **+**: In foreground - Indicates the process is in the foreground process group.
- **<:** Job control signal - Process has been modified by a job control signal.
- **N:** Low-priority - Indicates a process started with a nice value higher than 0.

Ps a: This command will print all processes running by all users.

```

bilal@bilal: ~
bilal@bilal:~$ ps a
  PID TTY          STAT       TIME COMMAND
 1826 tty2      Ssl+        0:00 /usr/lib/gdm3/gdm-x-session --run-script env GNOME_
 1828 tty2      Sl+         2:56 /usr/lib/xorg/Xorg vt2 -displayfd 3 -auth /run/user
 1853 tty2      Sl+         0:00 /usr/libexec/gnome-session-binary --systemd --syste
29917 pts/2      Ss+         0:00 bash
36976 pts/0      Ss          0:00 bash
37120 pts/0      R+          0:00 ps a
bilal@bilal:~$
  
```

Explanation: Here STAT shows the process state as described for the above output as below.

- Ssl+ for PID 1403 indicates that the process is interruptible sleep (waiting for an event to complete), is a session leader (s), is in the foreground process group (+), and is also multi-threaded (l).
- Ss+ for PID 1406 indicates that the process is interruptible sleep, is a session leader (s), and is in the foreground process group (+).
- Ss for PID 40869 indicates that the process is interruptible sleep and is a session leader (s).
- R+ for PID 40968 indicates that the process is running and is in the foreground process group (+).

Ps - a: Used to display all processes by all users except session leaders and process associated terminals such as bash. Xorg and gnome are related with graphics and desktop.

```
bilal@Bilal: ~  
bilal@Bilal:~$ ps -a  
  PID TTY          TIME CMD  
 1828 tty2      00:03:06 Xorg  
 1853 tty2      00:00:00 gnome-session-b  
 37766 pts/0      00:00:00 ps
```

Systemd: Systemd is also a session leader, to check execute `ps -j -p "1"` to display the first process ID along with session ID.

```
bilal@Bilal: ~  
bilal@Bilal:~$ ps -j -p "1"  
  PID PGID   SID TTY          TIME CMD  
    1    1     1 ?           00:00:04 systemd
```

Ps -A: Displays the entire processes running in the system.

```
bilal@Bilal: ~  
bilal@Bilal:~$ ps -A  
  PID TTY          TIME CMD  
    1 ?           00:00:03 systemd  
    2 ?           00:00:00 kthreadd  
    3 ?           00:00:00 rcu_gp  
    4 ?           00:00:00 rcu_par_gp  
    5 ?           00:00:00 slub_flushwq  
    6 ?           00:00:00 netns  
   10 ?           00:00:00 mm_percpu_wq
```

Ps u: Displays the CPU and memory usage.

```
bilal@Bilal: ~  
bilal@Bilal:~$ ps u  
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND  
bilal    1826   0.0   0.0 164360 6636 tty2    Ssl+  0:00   23:00 /usr/lib/gdm3/gdm-x-session --run-script e  
bilal    1828   0.1   0.7 672632 118936 tty2    Sl+   4:05   23:00 /usr/lib/xorg/Xorg vt2 -displayfd 3 -auth  
bilal    1853   0.0   0.0 188620 13856 tty2    Sl+   0:00   23:00 /usr/libexec/gnome-session-binary --system  
bilal    37279  0.0   0.0 11856 5780 pts/1    Ss   12:25   0:00 bash  
bilal    37330  0.0   0.0 11988 6304 pts/2    Ss   12:25   0:00 bash  
bilal    40074  0.0   0.0 11856 5984 pts/0    Ss+  13:14   0:00 bash  
bilal    40637  0.0   0.0 9840 3328 pts/2    S+   13:15   0:00 /bin/bash ./zotero  
bilal    40640  7.0   2.7 2584000 444848 pts/2    Sl+  13:15   0:08 /home/bilal/Downloads/Zotero-6.0.27_linux-x86_64/  
bilal    40810  0.0   0.0 11856 5856 pts/3    Ss+  13:15   0:00 bash  
bilal    40990  0.0   0.0 11832 3376 pts/1    R+   13:17   0:00 ps u
```

Top: Displays the Real-time processes with their PID, CPU, memory usage and other details. It shows the overall memory usage details at the top and individual process details.

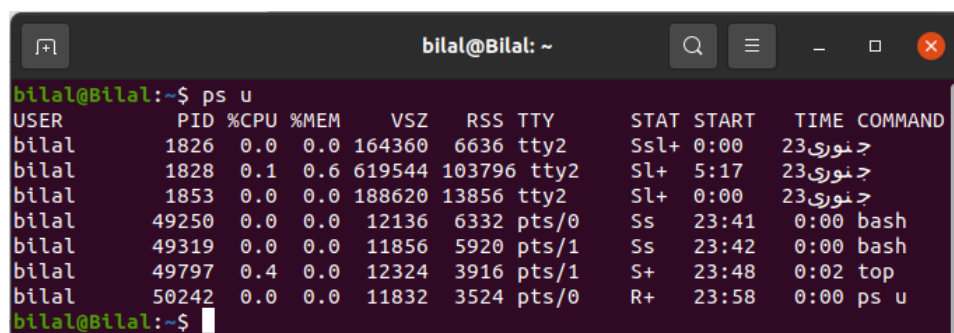
Summary:

```
bilal@Bilal: ~  
bilal@Bilal:~$ top  
  
top - 23:29:45 up 2 days, 23:11, 1 user, load average: 0.49, 0.57, 0.38  
Tasks: 321 total, 1 running, 320 sleeping, 0 stopped, 0 zombie  
%Cpu(s): 1.4 us, 0.7 sy, 0.0 ni, 97.8 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st  
MiB Mem : 15793.0 total, 5597.8 free, 6141.3 used, 4054.0 buff/cache  
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used, 8557.4 avail Mem
```

Individual Process Details:

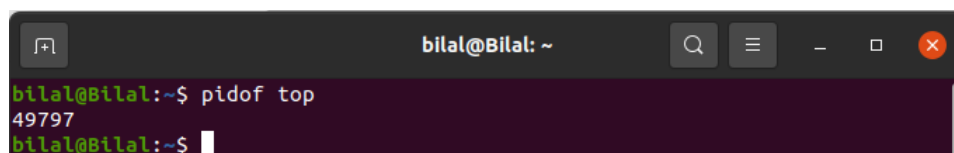
PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
6562	bilal	20	0	5018020	887048	226140	S	9.0	5.5	36:31.49	firefox
1981	bilal	20	0	4765728	290892	109472	S	8.3	1.8	8:42.89	gnome-s+
7645	bilal	20	0	3622188	958548	133436	S	4.7	5.9	25:26.08	Isolate+
1828	bilal	20	0	645168	120052	80484	S	3.7	0.7	4:53.51	Xorg
229	root	-51	0	0	0	0	S	3.3	0.0	0:50.88	irq/127+
1745	bilal	9	-11	4367732	22196	16880	S	2.3	0.1	14:36.79	pulseau+
12554	bilal	20	0	3264204	557360	138612	S	0.7	3.4	5:00.44	Isolate+
47457	bilal	20	0	2449704	109828	82772	S	0.7	0.7	0:04.55	Isolate+
496	root	-51	0	0	0	0	S	0.3	0.0	0:02.62	irq/134+
6986	bilal	20	0	2566372	127708	103600	S	0.3	0.8	0:17.02	Isolate+
10289	bilal	20	0	2956388	395656	132880	S	0.3	2.4	7:42.46	Isolate+
22617	bilal	20	0	3164068	503612	145660	S	0.3	3.1	2:57.80	Isolate+
47327	root	20	0	0	0	0	I	0.3	0.0	0:00.22	kworker+
48848	bilal	20	0	12324	3936	3284	R	0.3	0.0	0:00.17	top
1	root	20	0	168436	11724	8300	S	0.0	0.1	0:03.55	systemd
2	root	20	0	0	0	0	S	0.0	0.0	0:00.07	kthreadd
3	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	rcu_gp

Pidof: Displays the pid of a process using its command. Open another terminal and execute **ps u** to see the **top** command running as below.



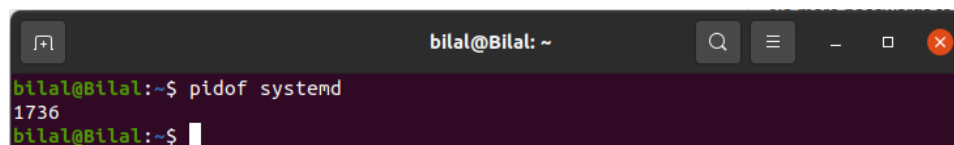
USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
bilal	1826	0.0	0.0	164360	6636	tty2	Ssl+	0:00	23:نوري	
bilal	1828	0.1	0.6	619544	103796	tty2	Sl+	5:17	23:نوري	
bilal	1853	0.0	0.0	188620	13856	tty2	Sl+	0:00	23:نوري	
bilal	49250	0.0	0.0	12136	6332	pts/0	Ss	23:41	0:00	bash
bilal	49319	0.0	0.0	11856	5920	pts/1	Ss	23:42	0:00	bash
bilal	49797	0.4	0.0	12324	3916	pts/1	S+	23:48	0:02	top
bilal	50242	0.0	0.0	11832	3524	pts/0	R+	23:58	0:00	ps u

Above you can see the top command with pid of 49497. And we can see the pid of top using pidof command.



49797

Systemd: Systemd is a Linux-specific system designed to operate within the Linux kernel. Serving as the initial process with a Process ID (PID) of 1, it is executed in user space as an integral part of the Linux startup sequence. Let's display the pid of systemd.



1736

It may be a subprocess of systemd having different id of systemd, let's use **ps -C systemd** to print all possible systemd ids running in the system.

```
bilal@Bilal: ~  
bilal@Bilal:~$ ps -C systemd  
  PID TTY          TIME CMD  
    1 ?           00:00:03 systemd  
 1736 ?           00:00:01 systemd  
bilal@Bilal:~$
```

Killing the process: In operating system we also need terminate or kill the processes for some reason like unresponsiveness and huge memory use of a process. We can do it by killing it.

```
CSS Copy code  
kill [signal] [PID]
```

Kill command is used to send a signal to a process, which can terminate it. Let's say there are two terminals (bash) that are open, and we want to close one of them. Below we can see we have two PIDs of bash and the one we are on has 8865.

```
bilal@Bilal:~$ pidof bash  
8865 4670  
bilal@Bilal:~$ ps  
  PID TTY          TIME CMD  
 8865 pts/1      00:00:00 bash  
 8912 pts/1      00:00:00 ps
```

Now let's terminate the bash having PID 4670 as below. After executing the following command, the other terminal will be terminated.

```
bilal@Bilal:~$ kill 4670
```

Signals: In Linux, signals are a fundamental inter-process communication mechanism. They are used to notify a process of a particular event, such as an interruption, termination, or user-defined event. We can print all signals along with their numbers by using **kill -l** command as below.

```
bilal@Bilal:~$ kill -l  
 1) SIGHUP      2) SIGINT      3) SIGQUIT     4) SIGILL      5) SIGTRAP  
 6) SIGABRT     7) SIGBUS      8) SIGFPE      9) SIGKILL     10) SIGUSR1  
11) SIGSEGV    12) SIGUSR2    13) SIGPIPE    14) SIGALRM     15) SIGTERM  
16) SIGSTKFLT  17) SIGCHLD   18) SIGCONT    19) SIGSTOP     20) SIGTSTP  
21) SIGTTIN    22) SIGTTOU    23) SIGURG     24) SIGXCPU     25) SIGXFSZ  
26) SIGVTALRM  27) SIGPROF   28) SIGWINCH   29) SIGIO        30) SIGPWR  
31) SIGSYS     34) SIGRTMIN  35) SIGRTMIN+1 36) SIGRTMIN+2 37) SIGRTMIN+3  
38) SIGRTMIN+4 39) SIGRTMIN+5 40) SIGRTMIN+6 41) SIGRTMIN+7 42) SIGRTMIN+8  
43) SIGRTMIN+9 44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12 47) SIGRTMIN+13  
48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13 52) SIGRTMAX-12  
53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9  56) SIGRTMAX-8  57) SIGRTMAX-7  
58) SIGRTMAX-6 59) SIGRTMAX-5 60) SIGRTMAX-4 61) SIGRTMAX-3 62) SIGRTMAX-2  
63) SIGRTMAX-1 64) SIGRTMAX  
bilal@Bilal:~$
```

The most common signal used to terminate a process is SIGTERM (signal number 15) which is by default. Some of the most commons are given below.

SIGTERM (15): Request to terminate a process.

SIGKILL (9): Terminate the process immediately like End Task in windows.

SIGSTOP (19): Pause a process.

SIGCONT (18): Resume a process.

Example 1: Below you can see we initiated the update process using apt and checked its PID in the second terminal. Then we stopped (paused) it using signal 19 and resumed it using signal 18. As in the first we can witness the whole process.

```
bilal@Bilal:~$ sudo apt update
Hit:1 http://packages.ros.org/ros/ubuntu focal InRelease
Hit:2 http://security.ubuntu.com/ubuntu focal-security InRelease
Err:3 http://pk.archive.ubuntu.com/ubuntu focal InRelease
  Temporary failure resolving 'pk.archive.ubuntu.com'
0% [Working]
[1]+  Stopped                  sudo apt update
Hit:4 http://pk.archive.ubuntu.com/ubuntu focal-updates InRelease
Hit:5 http://pk.archive.ubuntu.com/ubuntu focal-backports InRelease
Reading package lists... Done
Building dependency tree
Reading state information... Done
342 packages can be upgraded. Run 'apt list --upgradable' to see them.
W: Failed to fetch http://pk.archive.ubuntu.com/ubuntu/dists/focal/InRelease Temporary failure resolving 'pk.archive.ubuntu.com'
W: Some index files failed to download. They have been ignored, or old ones used instead.
bilal@Bilal:~$

bilal@Bilal:~$ pidof apt
13456
bilal@Bilal:~$ sudo kill -19 13456
[sudo] password for bilal:
bilal@Bilal:~$ sudo kill -18 13456
bilal@Bilal:~$
```

Example 2: Open a terminal, note its pid, pause and resume it from another terminal, and see the behavior of the first terminal opened.

Killing process by name: We can kill the process using its name using **killall** command as in below given example we opened multiple terminals and closed all using single entity (its name).

```
bilal@Bilal: ~
bilal@Bilal:~$ pidof bash
16015 15964 15913 15862 15811 15760 15707 8865
bilal@Bilal:~$ killall bash
bilal@Bilal:~$
```

Priority in Operating System: In any given moment, a Linux system hosts numerous processes, many initiated by the operating system itself, while others are generated by the logged-in user. Each process is assigned a priority by the system, dictating its execution speed. Generally, processes with higher priorities are executed before those with lower priorities. Priorities are 0 to 139 in which 0 to 99 for real-time and 100 to 139 for users.

Commands: Linux offers the 'nice' and 'renice' commands to modify a process's priority, thereby influencing its execution speed within the system.

Priority Types: Linux implements two types of priorities as given below.

Static Priority Range (NI ~ Niceness) (NI): Spans from -20 to 19, indicating the "niceness" of a process. Lower values signify higher priority, with 0 being the default. Processes with lower niceness values receive more CPU time, while those with higher niceness values receive fewer resources.

Dynamic Priority Range (PRI ~ Priority) (PRI): The priority value is the process's actual priority which is used by the Linux kernel to schedule a task. Ranges from 0 to 99 and is used for real-time tasks requiring immediate and predictable responses. Processes with higher dynamic priorities are scheduled with preference over those with lower priorities. The lower the value the higher the priority.

Example:

Static Priority Range (Niceness):

Suppose we have two processes, A and B, running on a Linux system. Process A has a nice value of -10, while process B has a nice value of 10.

Process A (nice value: -10) is considered to have a higher priority because it has a lower niceness value. It will receive more CPU time compared to other processes with higher niceness values.

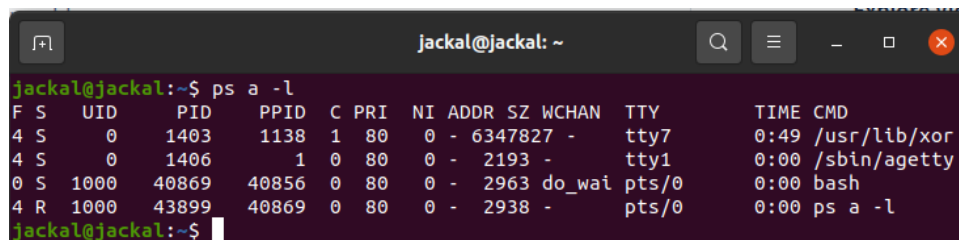
Process B (nice value: 10) is considered to have a lower priority because it has a higher niceness value. It will receive fewer CPU resources compared to other processes with lower niceness values.

Dynamic Priority Range (Real-time):

Let's say we have a real-time task, such as audio processing or network packet handling, running on the system with a dynamic priority of 90.

The real-time task with a dynamic priority of 90 has the highest priority among all processes on the system. It is scheduled preemptively, meaning it can interrupt lower-priority processes to execute immediately. This ensures that the real-time task receives immediate and predictable responses from the system, crucial for tasks like audio processing or network packet handling.

Checking the priority: To check priority we can use **ps -l** command as given below.



```
jackal@jackal: ~  
jackal@jackal:~$ ps a -l  
F S  UID  PID  PPID  C PRI  NI ADDR SZ WCHAN  TTY  TIME CMD  
4 S   0   1403   1138  1  80   0 - 6347827 -   tty7    0:49 /usr/lib/xor  
4 S   0   1406     1  0  80   0 - 2193 -   tty1    0:00 /sbin/agetty  
0 S 1000 40869 40856 0  80   0 - 2963 do_wai pts/0    0:00 bash  
4 R 1000 43899 40869 0  80   0 - 2938 -   pts/0    0:00 ps a -l  
jackal@jackal:~$
```

Explanation:

- **S:** The process state. This column indicates the current state of the process, such as running (R), sleeping (S), or waiting for input/output (D).
- **PRI:** The priority of the process. This column indicates the priority level assigned to the process by the system.

- **NI:** The nice value of the process. This column displays the "niceness" value of the process, which influences its priority.

Nice: Executing nice command without any arguments, it sets the process's static priority ID to 0.

```
jackal@jackal: ~
jackal@jackal:~$ nice
0
jackal@jackal:~$
```

Example – Step 1: Launch the python program as follows.

```
jackal@jackal:~$ python
Python 3.8.10 (default, Nov 22 2023, 10:22:35)
[GCC 9.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Step 2: Check the nice value.

```
jackal@jackal:~$ ps -l -a
F S  UID      PID     PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
0 S   1000    10875    6401  0  80   0 -  4902 do_sel pts/0    00:00:00 python
4 R   1000    10878    6453  0  80   0 -  2938 -      pts/1    00:00:00 ps
jackal@jackal:~$
```

Step 3: Close the Python and launch with desired nice value with command **nice -n -20 <command>** as sudo user.

```
jackal@jackal:~$ sudo nice -n -10 python
Python 3.8.10 (default, Nov 22 2023, 10:22:35)
[GCC 9.4.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Step 4: Check the nice value now.

```
jackal@jackal:~$ ps -l -a
F S  UID      PID     PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
4 S   0      11346    6401  0  80   0 -  3067 -      pts/0    00:00:00 sudo
4 S   0      11347    11346  0  70 -10 -  4838 -      pts/0    00:00:00 python
4 R  1000    11349    6453  0  80   0 -  2938 -      pts/1    00:00:00 ps
jackal@jackal:~$
```

Changing Niceness of running process: Renice can be used to change the niceness value of a running process with command **renice -n <PID>** as sudo user.

```
jackal@jackal: ~  
jackal@jackal:~$ sudo renice -n -20 11347  
11347 (process ID) old priority -10, new priority -20  
jackal@jackal:~$
```

Output:

```
jackal@jackal: ~  
jackal@jackal:~$ ps -l -a  
F S    UID      PID     PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD  
4 S    0       11346    6401   0  80   0  -  3067 -          pts/0      00:00:00 sudo  
4 S    0       11347    11346   0  60 -20  -  4838 -          pts/0      00:00:00 python  
4 R 1000    11908    6453   0  80   0  -  2938 -          pts/1      00:00:00 ps  
jackal@jackal:~$ ps -l -a
```

Exercise:

Task 1: Explore the following commands with examples.

- Pstree
- Lsof
- Watch
- Free
- Df
- Strace

Task 2: Write a simple script.sh file. Print some information, add delay (sleep) and track the process and its PID of script and sleep process using any tool.

Task 3: Write a simple python or C code, add delay, kill it before it gets completed and report the process.

Task 4: Write one simple script file and python file, check their NI and PRI, change them using nice and renice and report the process.